



Escuela de Ingeniería y Tecnologías

Proyecto de Introducción a la Ingeniería

Avances del Proyecto 1

Profesor: Antonio Martinez Torteya.

Geú Dávila Guerrero 656528
Angel Gabriel Romero Patena 694550
Esaúl Alejandro Villarreal Ábrego 646239

Este trabajo utilizó el apoyo de la inteligencia artificial de Google Gemini, limitado a la explicación de conceptos y problemas encontrados en el proceso de desarrollo (sin haberse especificado el código exacto, solo el concepto o problema general).

“Doy mi palabra que he realizado esta actividad con integridad académica.”

- Esaúl Alejandro Villarreal Ábrego
- Geú Dávila Guerrero
- Ángel Gabriel Romero Patena

Introducción

El objetivo de este trabajo es explorar el sistema I/O (Inputs y Outputs) del microcontrolador Arduino, así como la programación en C++, a través de un proyecto consistiendo de un juego donde un usuario presiona uno de tres botones disponibles dependiendo del color (rojo, verde o azul) que vea en un LED adyacente (Ver **Figura 1**). Al presionar el botón correcto, el LED se apaga y, una vez que se suelte el botón, un color nuevo se elige aleatoriamente. Ya sea que el usuario consiga o no presionar 10 veces el botón indicado dentro de un periodo de 30 segundos, le será indicado por un sonido de victoria o derrota generado por un buzzer pasivo (ver **Figura 1**).

Para comenzar, se explicará paso a paso cada uno de los procesos considerados tanto en el programa como en el circuito para lograr el funcionamiento adecuado, eficiente y óptimo del proyecto, desde la selección de colores para el LED hasta el control de intervalos al momento de realizar pitidos con el buzzer pasivo. En segundo lugar, se analizará cómo estos procesos se complementan e integran entre sí para establecer un sistema interactivo que permite al usuario participar en el juego descrito en el primer párrafo. Se realizará un análisis de las mejoras, dificultades, problemas y soluciones encontrados durante el proceso de desarrollo.

Metodología

Electrónica

A continuación se muestra el circuito electrónico que le permite al Arduino recibir (Input) y emitir (Output) información relacionada al caso definido en la introducción, logrando el objetivo de explorar el sistema I/O del microcontrolador.

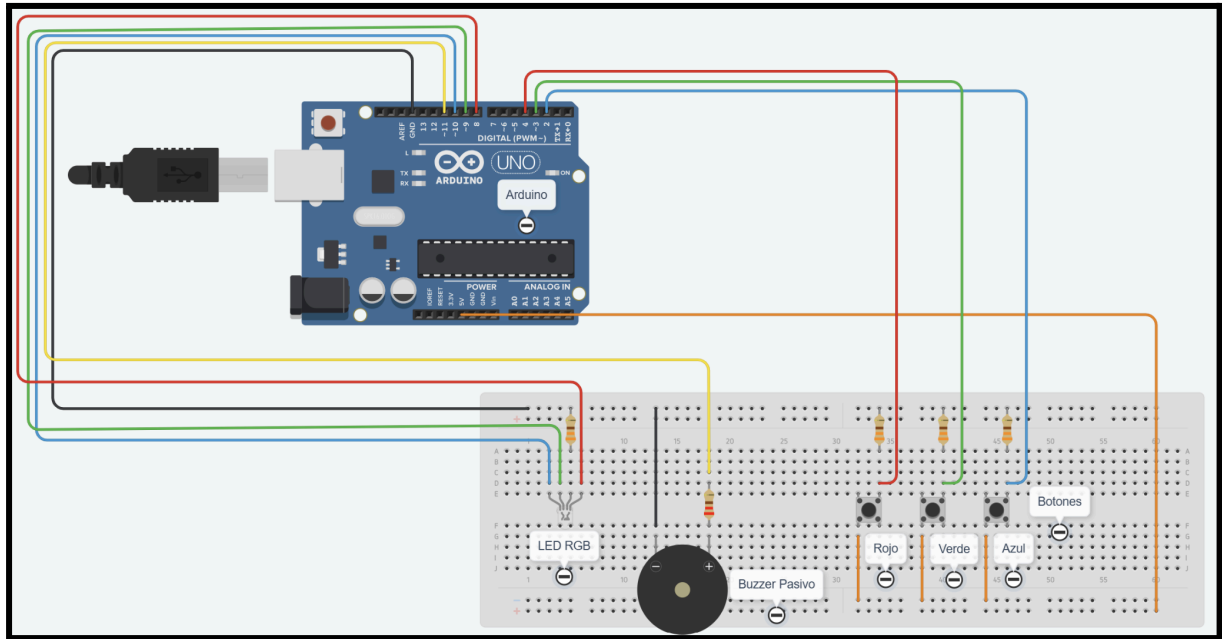


Figura 1: Diagrama de las conexiones del sistema utilizado, incluyendo descripciones de los componentes principales del circuito electrónico (no se utilizó el Arduino Mega utilizado en el circuito verdadero dado a que no había un modelo disponible). [Diagrama hecho con Tinkercad]

La sección de INPUT consta de 3 botones, cada uno asignado un color (rojo, verde y azul). La sección de OUTPUT consta de un LED RGB y de un Buzzer Pasivo. A continuación, en la **Tabla 1**, se explican y justifican los componentes y conexiones utilizados.

Componente	Conectado con	Uso/Justificación	Cantidad
LED RGB de cátodo común.	Pines digitales del Arduino 8 (rojo), 9 (verde) y 10 (azul), permiten mandar 5V de manera individual para prender el LED ya sea en rojo, verde o azul. Se conecta la pata más larga a tierra a través de un resistor (Google, 2026a).	Indica al usuario el color del botón que debe presionar. Se utilizó en vez de tres LEDs convencionales para ahorrar espacio y simplificar las conexiones requeridas (usando 4 cables en vez de 6, y solo 1 resistor en vez de 3).	1
Buzzer Pasivo	Pin digital 11 (PWM)	Permite emitir distintos tonos para crear una melodía de victoria o de derrota (dependiendo del puntaje obtenido por el jugador). Se utiliza un pin digital PWM para mandar las señales variadas requeridas para generar los tonos (Google, 2026b).	1
Botones Push	Conectados a los pines digitales del Arduino 2 (azul), 3 (verde) y 4 (rojo).	Le permiten al Arduino detectar qué color seleccionó el usuario al dejar pasar corriente hacia los pines digitales correspondientes.	3
Resistores de 330 Ω	Al ser el LED RGB de cátodo común, la resistencia se utilizó en la pata del cátodo para conectar el LED a tierra. Los resistores se conectaron de manera diagonal a los cables que suministran 5 V para asegurar que el circuito se cierre solo cuando se presione el botón.	LED RGB: de acuerdo con IC Components Limited (s.f.) la corriente ideal para este componente es de 20 mA (Requisitos de potencia y voltaje para LED RGB). A 5V (suministrados por los pines del Arduino), el resistor permite mantener la corriente recibida por el LED en aproximadamente 15 mA, dentro de su rango seguro de operación. Botones: Conectan los botones a tierra, permitiendo que al ser presionados cierren el circuito sin cortocircuitos (Google, 2026c). Esto, a su vez, permite mandar el input correspondiente al Arduino. Los resistores también previenen que interferencia electromagnética genere inputs falsos para el Arduino cuando el circuito está abierto.	4
Resistor de 220 Ω	Conecta el buzzer pasivo con la tierra.	De acuerdo con Shenghui Electronics (2025), el rango de corriente bajo el cual puede operar un buzzer pasivo ronda entre los 5 mA y los 30 mA (párr. 19). En el caso de que se mande el voltaje máximo de 5V que puede producir el pin 11 (ver	1

		conexiones de buzzer pasivo), la corriente quedaría limitada a aproximadamente 23 mA, dentro del rango de operación seguro del buzzer.	
--	--	--	--

Tabla 1: Lista de componentes y conexiones utilizados en el circuito y la justificación de su uso. [Tabla de autoría propia]

Programación

Cumpliendo con el segundo objetivo de este proyecto, la exploración de la programación en C++, se realizó un programa en Arduino IDE que cumple con las siguientes funcionalidades:

- Mantener un registro del tiempo limite, del tiempo transcurrido y de la cantidad de aciertos hechos por el usuario dentro del tiempo límite.
- Definir ya sea una victoria o una derrota dependiendo de la cantidad de aciertos hecha por el usuario dentro del tiempo límite (mínimo 10 para considerarse victoria).
- Emitir una melodía, mediante el buzzer pasivo, que caracteriza una victoria o una derrota.
- Elegir de manera aleatoria entre el rojo, el verde y el azul y asignarle el color elegido al LED RGB.
- Mantener al LED encendido mientras no se realice ningún INPUT (presionar uno de los botones) y mantenerlo apagado mientras se mantenga presionado el botón correcto.
- Mantener un flujo constante y preciso del tiempo transcurrido que no es interrumpido por los INPUTS.
- Avisar al usuario, mediante el buzzer pasivo, cuando quede la mitad del tiempo límite y cuando queden 5 o menos segundos.
- Reiniciar el juego, avisando al usuario mediante el buzzer pasivo, una vez que se haya concluido la partida.

En las primeras líneas del código se definieron variables globales para tomar registro del tiempo límite y restante, la cantidad de aciertos realizada, los pines digitales utilizados y demás (ver comentarios en **Figura 2**). A su vez, se definieron de manera preliminar las funciones adicionales a *setup* y *void* que fueron utilizadas para muchas de las funcionalidades del programa previamente descritas (ver comentarios en **Figura 2**).

```

1  int time; //Contador de tiempo restante.
2  int score; //Contador de aciertos.
3  int t = 30; //Usado de referencia para saber cuándo ha pasado la mitad del tiempo límite.
4
5  int p; //Variable que evita que se presiona el botón correspondiente más de una vez por ronda.
6
7  int r = 8; //Pin digital 8 enciende rojo.
8  int g = 9; //Pin digital 9 enciende verde.
9  int b = 10; //Pin digital 10 enciende azul.
10
11 int b1 = 4; //Pin digital que recibe señal del botón correspondiente al rojo.
12 int b2 = 3; //Pin digital que recibe señal del botón correspondiente al verde.
13 int b3 = 2; //Pin digital que recibe señal del botón correspondiente al azul.
14 int s; //Variable que define qué botón se espera que sea presionado al momento.
15
16 int buzz = 11; //Pin 11 asignado al buzzer pasivo.
17
18 void victory(); //Función que realiza sonido de victoria.
19
20 void defeat(); //Función que realiza sonido de derrota.
21
22 void countdown(); //Función que realiza sonido de reinicio de juego.
23
24 void warning(); //Función que se encarga de alertar al usuario cuando haya pasado la mitad del tiempo límite y cuando queden 5 o menos segundos.
25

```

Figura 2: Sección del programa donde se definieron variables globales y funciones adicionales a *setup* y *void*. [Imagen de autoría propia]

En el void *setup* (ver **Figura 3**) se inició comunicación serial entre la computadora y el arduino para verificar (a través de la impresión de las variables *time* y *score*) el funcionamiento apropiado del programa y del circuito. A su vez, de acuerdo a aquello mencionado por Google (2026d), se utilizó la función *randomSeed* para evitar que la generación de números repetitivos en vez de aleatorios en el programa (párr. 5). Se tomó como argumento de esta función la interferencia eléctrica detectada (mediante la función *analogRead*) en el pin análogo A0 (Google, 2026d, párr. 6). Se hace esto dado a que es un valor aleatorio y no constante. Finalmente, se utilizaron las variables definidas previamente para definir, a través de la función *pinMode*, INPUTS conectados a los botones y OUTPUTS conectados al buzzer pasivo y al led RGB.

```

26 void setup() {
27     Serial.begin(9600); //Iniciar comunicaciones para monitoreo de funcionalidad.
28
29     randomSeed(analogRead(A0)); //Inicializa el generador de números aleatorios para evitar que sea predecible.
30
31     //Se definen a los pines que activan el LED como OUTPUTS.
32     pinMode(r, OUTPUT);
33     pinMode(g, OUTPUT);
34     pinMode(b, OUTPUT);
35
36     //Se define el pin utilizado para el buzzer pasivo como OUTPUT.
37     pinMode(buzz, OUTPUT);
38
39     //Se definen a los pines que detectan activaciones de botones como INPUTS.
40     pinMode(b1, INPUT);
41     pinMode(b2, INPUT);
42     pinMode(b3, INPUT);
43 }
44

```

Figura 3: La función *setup* comienza comunicación entre el arduino y la computadora y asigna pines como INPUTS y OUTPUTS. [Imagen de autoría propia]

Para realizar cada partida, se utiliza código en la función *loop*, permitiendo que el juego sea repetido cuantas veces sea deseado. Para cada partida, se reinicia el tiempo restante (tomando en cuenta el tiempo límite previamente asignado) y la cantidad de

aciertos hecha por el usuario (ver **Figura 4**). Esto evita que partidas pasadas tengan una influencia sobre la partida actual.

Se utiliza un bucle de tipo *while* (mostrado a lo largo de la **Figura 4 a 7**) para ejecutar el procedimiento del juego mientras todavía quede tiempo restante. En primer lugar, se apaga completamente el LED RGB (mediante la función *digitalWrite*) para evitar que, al cambiar de color, se combine el color actual con el pasado de tal forma que el usuario no sepa qué botón presionar (e.j. azul se combina con rojo para generar morado). Después, se elige un número entre 0 y 2 con la función *random* (Google, 2026d, párr. 4). Esto permite elegir aleatoriamente entre los colores rojo, verde y azul. Una vez elegido el color, se usa un *switch case* (Google, 2026e). Esto permite:

- Encender el color correspondiente en el LED RGB (a través de la *digitalWrite*).
- Asignar (a través de la variable *s*) el botón que el programa debe esperar que sea presionado para seguir adelante y contar un acierto.

```
45 void loop() {
46   time = t+1; //Se reinicia el tiempo límite para cada partida. Se toma en cuenta fenómeno observado donde se le resta 1 al tiempo asignado.
47   score = 0; //Reinicia número de aciertos para cada partida.
48   while (time > 0) {
49     //Reinicia el LED para evitar mezclas de colores.
50     digitalWrite(r, LOW);
51     digitalWrite(g, LOW);
52     digitalWrite(b, LOW);
53
54     //Se asigna de manera aleatoria un color al LED y el botón del color correspondiente.
55     switch (random(3)) { //La función random(3) genera un número entero aleatorio entre 0 y 2.
56       case 0: //Cada caso está asignado a un posible valor de random(3).
57         digitalWrite(r, HIGH); //Asigna color al LED
58         s = b1; //Se asigna el botón (correspondiente al color asignado()) que debe ser presionado.
59         break; //Evita que se ejecuten los otros casos.
60       case 1:
61         digitalWrite(g, HIGH);
62         s = b2;
63         break;
64       case 2:
65         digitalWrite(b, HIGH);
66         s = b3;
67         break;
68     }
69 }
```

Figura 4: primera sección de la función *loop*. Se reinician los valores de *time* y *score*, se apaga el LED previo a la asignación de un color y se asigna este último (y el botón que debe ser presionado) de manera aleatoria. [Imagen de autoría propia]

Se incluye un *while* loop adicional (ver **Figura 5**) para que el programa para que no avance el programa sin que el usuario haya presionado el botón correspondiente (*digitalRead(s) != HIGH*). Se utiliza la función *warning* para asegurar que ha pasado 1 segundo por cada reducción por 1 (*time*–) que se realiza en el tiempo restante. La función también chequea si ya ha pasado la mitad del tiempo restante o si quedan 5 o menos segundos para avisar al usuario a través de un pitido de 392 Hz usando el buzzer activo.

Después, el programa envía la información del tiempo restante en el monitor serial (*Serial.println*), la cual fue utilizada para ver si hay algún error en la velocidad o en la magnitud en la reducción del tiempo restante. Finalmente, para prevenir la posibilidad de que el programa siga esperando un INPUT incluso cuando el tiempo se ha terminado, se utiliza una condicional *if* que realiza un *break* para salir del loop y seguir adelante.

Al probar el programa se hizo la observación de que al presionar el botón se podían llegar a registrar dos aciertos seguidos, se incluyó la variable *p* para asegurar que el

programa solo otorgue un acierto a la vez. En la **Figura 5**, se le asigna a *p* un valor de 0, el cual significa que todavía no ha sido presionado el botón.

```
70 //Evita que el programa continúe sin que se haya presionado el botón correspondiente al color del LED.
71 while (digitalRead(s) != HIGH) {
72     p = 0; //Permite que el botón pueda volver a ser presionado.
73     warning(); //Ver descripción al inicio del código.
74     time--; //Va reduciendo el contador de tiempo restante.
75     Serial.println(time); //Permite monitorear el tiempo desde el ordenador.
76
77     if (time == 0) { //Evita que el programa se atore en caso de que, mientras se espere que se presione el botón, el tiempo se acabe.
78         break;
79     }
80 }
81
```

Figura 5: segunda sección de la función *loop*. Se lleva a cabo un registro del tiempo restante y se espera a que el usuario presione el botón indicado. [Imagen de autoría propia]

Cuando el jugador haya presionado el botón indicado, o se acabe el tiempo, el *while* loop deja de estar activo y permite la continuación al código mostrado en la **Figura 6**, el cual asigna aciertos. Se hace uso de una condicional de tipo *if*. Si no se ha acabado el tiempo y todavía no se tiene registro de que el botón ha sido presionado previamente, se le asigna un acierto al usuario (*score++*) y se cambia el valor de *p* a 1 (indicando que ya ha sido presionado el botón y no se deben de registrar más aciertos). No se utiliza un delay para darle tiempo al programa de registrar que ya ha sido presionado el botón dado a que esto afectaría el flujo del tiempo restante.

```
82 //Si el tiempo ya se acabó o ya se presionó el botón antes de que quede registrado en el Arduino, se previene que se asigne un acierto adicional indebido.
83 if (time != 0 && p == 0)
84 {
85     score++; //Incrementa el registro de aciertos por uno.
86     p = 1; //Evita que se registre más de una presionada de botón por ronda.
87 }
88
```

Figura 6: tercera sección de la función *loop*. Se lleva a cabo el registro de la cantidad de aciertos con base en si todavía queda tiempo y si el botón no ha sido presionado previamente. [Imagen de autoría propia]

Finalmente, para asegurar que el programa cumple con las especificaciones de la tarea, se creó un último *while* loop (ver **Figura 7**) para mantener al LED apagado (*digitalWrite*) mientras que se tenga presionado el botón correcto. Al inicio no se entendió esta funcionalidad hasta que se observó que no tener esta sección del código permitía al usuario obtener una gran cantidad de aciertos sin ningún tipo de reto, removiendo la diversión del juego. Este loop es muy similar a aquel visto en la **Figura 5**, haciendo el mismo uso de la función *warning* y de *time--* así como la condicional para prevenir esperar *Inputs* una vez que se acabe el tiempo. Sin embargo, ahora la condición utilizada es que se tenga presionado el botón de manera continua (*digitalRead(s) == HIGH*) y se mantiene al LED completamente apagado (mediante *digitalWrite*).


```

89 //Mantiene al LED apagado mientras se tenga presionado el botón, haciendo que el tiempo de la partida continúe sin interrupciones.
90 while (digitalRead(s) == HIGH) {
91     warning();
92     time--;
93     Serial.println(time);
94     digitalWrite(r, LOW);
95     digitalWrite(g, LOW);
96     digitalWrite(b, LOW);
97     if (time == 0) {
98         break;
99     }
100 }
101 }
102

```

Figura 7: cuarta sección de la función *loop*. Se mantiene el LED apagado mientras que el usuario siga presionado el botón correcto. [Imagen de autoría propia]

Tras salir del primer *while* loop y acabarse la partida (ver **Figura 8**), se utiliza *Serial.print* y *Serial.println* para imprimir la cantidad de aciertos finales hechos por el usuario (permitiendo así verificar que no se hayan dado aciertos de más o de menos durante la partida). Después, se apaga el LED (*digitalWrite*) para que el usuario sepa que ha acabado el tiempo. Tras el pasar de 3 segundos, se utiliza el buzzer pasivo para reproducir una melodía de victoria (función *victory*) o de derrota (función *defeat*) dependiendo de si el jugador adquirió o no al menos 10 aciertos (condicional *if*). Se realiza una espera de 3 segundos dado a que se identificó que, sin esta, se confunden los últimos pitidos realizados por el buzzer al quedar 5 o menos segundos en el tiempo restante con los primeros pitidos de las melodías. Finalmente, se vuelve a hacer otra espera de 3 segundos para no confundir los pitidos de las melodías con el pitido generado por la función *countdown*, la cual genera 10 pitidos de velocidades cada vez mayores para que el jugador tenga tiempo suficiente para prepararse para la siguiente partida.

```

103 //Permite conocer cantidad de aciertos totales hechos durante la partida.
104 Serial.print("score = ");
105 Serial.println(score);
106
107 //Apaga foco LED para evitar que el usuario piense que la partida continúa cuando en realidad ya se acabó.
108 digitalWrite(r, LOW);
109 digitalWrite(g, LOW);
110 digitalWrite(b, LOW);
111
112 //Da tiempo suficiente para crear un espacio entre el sonido hecho cuando quedan 5 o menos segundos y el sonido ya sea de victoria o derrota.
113 delay(3000);
114
115 //Revisa la cantidad de aciertos finales hechos en la partida y, con base en eso, emite el sonido ya sea de victoria (10 o más aciertos) o de derrota (menos de 10 aciertos).
116 if (score >= 10) {
117     victory(); //Ver descripción al inicio del código.
118 }
119 else {
120     defeat(); //Ver descripción al inicio del código.
121 }
122
123 //Permite al jugador prepararse para la siguiente ronda.
124 delay(3000);
125 countdown(); //Ver descripción al inicio del código.
126 }
127

```

Figura 8: última sección de la función *loop*. Se muestra la cantidad de aciertos finales, se apaga el LED, se indica la victoria o derrota a través del buzzer pasivo y después se realiza una serie de pitidos para advertir el comienzo de la siguiente partida. [Imagen de autoría propia]

A continuación, en la **Figura 9**, se muestra el código de las funciones *victory*, *defeat*, *countdown* y *warning* utilizados en la función *loop*. Cabe destacar el uso de las funciones de *tone*, *delay* y *noTone* para generar las frecuencias y duraciones necesarias para realizar las melodías y pitidos requeridos. Se hizo uso de inteligencia artificial para generar las melodías de victoria y derrota (Google, 2026f; Google, 2026g). Al notar que las melodías no se

escuchaban bien, se modificó el tiempo de reproducción de cada tono así como el tiempo de espera entre tonos.

En la función *countdown*, se puede observar el uso de un *for* loop para variar el tiempo de espera entre cada pitido a lo largo de los 10 pitidos. A su vez, la función *warning* utiliza condicionales *if* para ver cuando queda la mitad del tiempo y 5 o menos segundos. Utiliza esta información para saber cuándo activar el buzzer pasivo y reducir el tiempo de espera entre cada reducción del tiempo restante (*time--*) a 800 ms en vez de 1000 ms para tomar en cuenta el tiempo que le toma al buzzer hacer el pitido.

```
128 void victory() {
129     tone(buzz, 392, 500);
130     delay(200);
131     noTone(buzz);
132     tone(buzz, 392, 500);
133     delay(200);
134     noTone(buzz);
135     tone(buzz, 392, 500);
136     delay(200);
137     noTone(buzz);
138     tone(buzz, 311, 1000);
139     delay(500);
140     tone(buzz, 349, 500);
141     delay(200);
142     tone(buzz, 392, 1500);
143     delay(200);
144 }
145
146 void defeat() {
147     tone(buzz, 330, 500);
148     delay(200);
149     tone(buzz, 311, 500);
150     delay(200);
151     tone(buzz, 294, 500);
152     delay(200);
153     tone(buzz, 277, 1000);
154 }
155
156 void countdown() {
157     //El sonido se va repitiendo cada vez más rápido a lo largo de 10 repeticiones.
158     for (int i = 10; i > 0; i--) {
159         tone(buzz, 392, 500);
160         delay(i*100);
161         noTone(buzz);
162     }
163 }
164
```

```
165 void warning() {
166     //Manda señal al buzzer para que suene.
167     if (time == round(t/2) || time <= 5) {
168         tone(buzz, 392, 500);
169         delay(200);
170     }
171
172     //Toma en cuenta el tiempo que toma hacer sonar el buzzer (200 ms) cuando ha pasado la mitad del tiempo límite o queden 5 o menos segundos.
173     if(time <= 5 || time == round(t/2)) {
174         delay(800); //Junto con el tiempo que suena el buzzer llega a 1000 ms.
175     }
176     else {
177         delay(1000);
178     }
179 }
```

Figura 9: los códigos usados para crear las funciones de *victory*, *defeat*, *countdown* y *warning*. [Imagen de autoría propia]

Resultados

Tras realizar varias revisiones del código, se consiguió grabar un video donde se demuestran las funcionalidades mencionadas al inicio de la sección de **Programación**. Cabe destacar que el video fue realizado previo a la adición del resistor de $220\ \Omega$ al circuito para proteger al buzzer pasivo, por lo cual los pitidos escuchados son más fuertes que aquellos que la versión más reciente del circuito realiza. Sin embargo, esta es la única diferencia y, por ende, se decidió usar ese video en vez de realizar uno nuevo, donde además se podría tener mayor dificultad escuchando los pitidos emitidos. El video se puede encontrar adjunto a este documento en la entrega.

Conclusión

En conclusión, este proyecto representó un reto con múltiples partes funcionales, lo cual permitió reforzar la habilidad de análisis tanto de circuitos como de código y la relación entre sí. A futuro, se podría mejorar el proyecto al incluir un sistema de audio más avanzado que permita reproducir voces (lo cual podría servir para dar indicaciones al usuario). A su vez, se podría implementar un método para evitar un bug detectado en el cual, al ser presionados los tres botones de manera simultánea, se detiene el programa. No se implementó ninguna corrección dado a la complejidad general del proyecto y dado a que no representaba una ventaja injusta para el usuario.

Referencias

Google. (2026a). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=common+cathode+rgb+led&oq=common+cathode+rgb+led&gs_lcrp=EgZjaHJvbWUyBggAEFEUYOTIHCAEQABiABDIICAIQABgWGB4yCAgDEAAyFhgeMggIBBAAGBYHjIICAUQABgWGB4yCAgGEAAyFhgeMgolBxAAGAoYFhgeMggICBAAGBYHjIIcAKQABgWGB7SAQg3NzQ2ajBqN6gCALACAA&sourceid=chrome&source=chrome.ob&ie=UTF-8

Google. (2026b). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=how+to+use+passive+buzzer+arduino&sca_esv=79a7fd24df7232ff&sxsrf=APpeQnvoMh-k8eHnNZVXd4Do_ecasb0LYw%3A1787446887319&ei=Z0aKap6KE5ymqtsP8YuM6Qk&biw=1707&bih=898&ved=0ahUKEwje2ZfGxrWWAxUck2oFHfEFI50Q4dUDCBA&uact=5&oq=how+to+use+passive+buzzer+arduino&gs_lp=Egxnd3Mtd2l6LXNlcniAilWhvdyB0byB1c2UgcGFzc2l2ZSBidXp6ZXIlgYXJkdWlubzIIEAAyGAQyBhAAGAgYHjIIIEAAyGAQYigUYhgMyCxAAGIAEGloFGIYDMgsQABiABBiKBRiGAzIIIEAAyGAQYogQyCBAAGIAEGKIEMggQABiABBiBDIFEAAY7wUyBRAAGO8FSO8ZUM0IWJIZcAN4AZABAJgBYaABngmqAQIxnrgBA8gBAPgBAZgCEaACmQjCAgoQABhHGNYEGLADwglGEAAyBxgewglHEAAyGAQYDclCCBAAGAgYBxgemAMAiAYBkAYIkgcCMTegB71isgcCMTS4B5MIwgcEMS4xNsgHGoAlAQ&sclient=gws-wiz-serp

Google. (2026c). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=why+is+my+circuit+giving+high+to+pin+on+arduino+even+when+I+haven%27t+pressed+the+button&oq=why+is+my+circuit+giving+high+to+pin+on+arduino+even+when+I+haven%27t+pressed+the+button&gs_lcrp=EgZjaHJvbWUyBggAEEUYOdIBCTE4NzAwajBqN6gCALACAA&sourceid=chrome&source=chrome.ob&ie=UTF-8

Google. (2026d). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=random+library+arduino&oq=random+library+arduino&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTINCAEQABiGAxiABBikBTINCAIQABiGAxiABBikBTIKCAMQABiiBBiJBdIBCDMzNzZqMGo3qAIAA&sourceid=chrome&source=chrome.ob&ie=UTF-8

Google. (2026e). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=switch+statements+arduino&oq=switch+statements+arduino&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIICAEQABgWGB4yCAgCEAAyFhgeMggIAxAGBYHjIICAQQABgWGB4yCAgFEAAyFhgeMggIBhAAGBYHjIICAQABgWGB4yCAgIEAAyFhgeMggICRAAGBYHjIICDQwMjhgMGo3qAIAA&sourceid=chrome&source=chrome.ob&ie=UTF-8

Google. (2026f). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=victory+sound+effect+final+fantasy+translated+to+frequencies+and+times+for+arduino+buzzer&sca_esv=79a7fd24df7232ff&biw=1707&bih=898&sxsrf=APpeQnsSpGxpzu-xDrnC3duWDUSyNTXbxA%3A1787447363574&ei=Q0iKaunMlp21qtsPw5jVoQ4&ved=0ahUKEwip-qOpyLWWAxWdmmoFHUNMNeQQ4dUDCBA&uact=5&oq=victory+sound+effect+final+fantasy+translated+to+frequencies+and+times+for+arduino+buzzer&gs_lp=Egxnd3Mtd2l6LXNlcniAiwXZpY3Rvcnkgc291bmQgZWZmZWNOIGZpbmFslGZhbniRhc3kgdHJhbnNsYXRIZCB0byBmcmVxdWVudY2lcyBhbmQgdGlZXMgZm9yIGFyZHVpbm8gYnV6emVySIQYUkwKWO8WcAF4AZABAjgBTqABgQeqAQIxlNLgBA8gBAPgBAZgCAqACQ8ICChAAGEcY1gQYsAPCAgQQIRgKmAMAIAYBkAYIkgcBMqAHuR-yBwExuAdAwgcDMS4xyAcCgAgB&sclient=gws-wiz-serp

Google. (2026g). *Gemini* (versión actual) [modelo de lenguaje de gran tamaño].

https://www.google.com/search?q=defeat+sound+effect+for+passive+buzzer+arduino&sca_esv=79a7fd24df7232ff&biw=1707&bih=898&sxsrf=APpeQnvSHWRnABGFdXfMtVnWFdmHO3k3MA%3A1787448581237&ei=BU2KavCJDv6qgtsP78b2sQ8&ved=0ahUKEwiwoPTtZLWWAxV-IWoFHW-jPfyQ4dUDCBA&uact=5&oq=defeat+sound+effect+for+passive+buzzer+arduino&gs_lp=Egxnd3Mtd2l6LXNlcniAilMhRIZmVhdCBzb3VuZCBIZmZlY3QgZm9yIHhBhc3NpdmUgYnV6emVyIGFyZHVpbm8yBRAhGJ8FSMYjULEBWPAhcAF4AZABAjgBTqABYw2qAQIyN7gBA8gBAPgBAZgCHKAC_g3CAgoQABhHGNyEGLADwglFEAAyGATCAgYQABgWGB7CAggQABgWGB4YCsICCxAAGIAEGloFGIYDwglIEAAyGAQYogTCAGUQABjvBclCBBAhGBWYAwClBgGQBgiSBwlyOKAHv0yyBwlyN7qH_A3CBwQyMS43yAcYqAgB&sclient=gws-wiz-serp

IC Components Limited. (s.f.). *Todo lo que necesita saber sobre los LED RGB*. IC Components Limited.
<https://www.ic-components.es/blog/Everything-You-Need-to-Know-About-RGB-LEDs.jsp>

Shenghui Electronics. (2025, 17 de noviembre). *¿Cuántos voltios necesita un zumbador piezoeléctrico?*. Shenghui Electronics.
<https://www.shenghui.com/es/How-many-volts-does-a-piezo-buzzer-need.html>